Andrew Pols

(519) 818-1374 | andrew.pols@mail.utoronto.ca | github.com/andrewpols
andrewpols.com | linkedin.com/in/andrewpols

Education

University of Toronto Sep. 2024 - Aug. 2028

- HBSc., **Computer Science and Statistics** | **GPA: 4.00 / 4.00**
- Relevant coursework: Software Design; Computer Organization; Software Tools and Systems Programming

Experience

Software Developer Intern May 2026 - May 2027

IBM Canada Ltd.

- Shipped a dependency graph for IBM Planning Analytics Workspace in React and TypeScript, so clients can trace how financial models connect. Kept it responsive on 2,500+ objects by rendering only the relevant subset.
- Cut a manual trace across thousands of objects from hours to minutes by traversing relationships with lazy-loading, caching, and concurrent asynchronous requests to the REST APIs of TM1, IBM's in-memory database.
- Collaborated with design, product management, QA, and engineering in Agile sprints to define graph layout, filtering, and object-specific interactions for large dependency structures, then shipped the feature to clients.
- Sped up development by 45% using IBM Bob, an AI coding agent, to run code reviews and generate test cases.

Projects

ScholarSearch | *Source Code* **Python | FastAPI | PostgreSQL | ChromaDB | Redis | ReactJS**

- Built a full-stack research paper search engine over 1,000,000+ papers, using PostgreSQL for persistence and ChromaDB for vector embeddings, with Redis caching to serve semantic search queries via FastAPI in under 200ms.
- Constructed D3.js citation graphs so researchers can view a paper's impact and related work in an interactive page.
- Containerized Postgres, ChromaDB, and Redis with Docker so the whole team shared a consistent development setup.

Multiplayer Pokémon Battle Server | *Source Code* **C | TCP Sockets | UNIX | ncurses**

- Built a multiplayer Pokémon battle server in C that pairs players into 1v1 matches, serving many concurrent TCP clients from a single-threaded event loop (select) with a lobby, rematches, forfeits, live chat, and an ncurses UI.
- Defined a binary packet protocol with a 5-byte header (command byte plus length in network byte order), buffering reads so fragmented TCP packets parse correctly and cleaning up matches and notifying opponents on disconnects.
- Implemented a generic doubly linked list to track clients and active matches, added timestamped connection and match logging, and eliminated memory errors and use-after-free bugs using Valgrind and AddressSanitizer.

FindMySound | *Source Code* **Python | Django | MySQL | Scikit-Learn | ReactJS**

- Built a web app that recommends music based on a user's music library, using Spotify's Web API and OAuth 2.0.
- Applied a KNN model on ingested BERT feature vectors to recommend music, reaching 95% user satisfaction.
- Stored user accounts in MySQL through a custom Django user model with JWT-based authentication, and exposed RESTful endpoints for authentication, account syncing, and music recommendations to the React frontend.

GroupFlow | *Source Code* **Java | MongoDB | Swing**

- Built a Java desktop app with Swing and MongoDB where students schedule meetings and assign group tasks.
- Followed Clean Architecture and SOLID design principles, with each use case in its own isolated package.
- Collaborated in a team of 6, enforcing CheckStyle and requiring thorough PR reviews before merging code.

Technical Skills

- **Programming Languages**: Java, Python, C, TypeScript, JavaScript, SQL, HTML5, CSS
- **Frameworks & Libraries**: React, Next.js, FastAPI, Django REST, Flask, Node.js, Scikit-learn, OpenCV
- **Databases**: PostgreSQL, MySQL, MongoDB, Redis, ChromaDB
- **Developer Tools**: Git, Docker, AWS, Azure, UNIX, Valgrind, AddressSanitizer, Nginx, Jira